

iOSDC Japan 2026 ・ Regular Talk ・ 40 min

映像変換サーバーなしで iPhone端末内で HLSを生成してライブ配信

Hikaru Sato @SatoHikaruDev

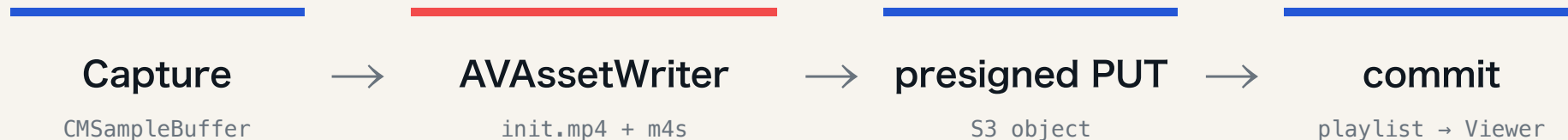
playlist.m3u8

init.mp4

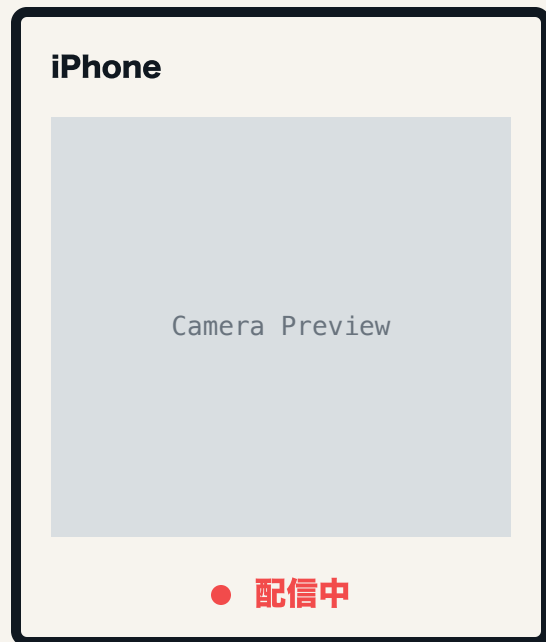
000001.m4s

— TODAY'S RESULT

iPhoneが生成したHLSを 撮影中からS3へ公開する



実機デモ：撮影しながら、ブラウザで追従再生する



HLS → S3

start	+2s	after PUT
PUT init.mp4	PUT seg 1	commit

CloudFront / Viewer

LIVE +4.2s

playlist.m3u8を追従再生

撮影後に待つほど、 共有したい瞬間から遠ざかる

子どもの動画を家族へ送る。

撮影中から届き始めれば、終了後の待ち時間を減らせる。

撮影終了後

recording.mov

大きな1ファイルを送信



撮影中

init.mp4

001.m4s

002.m4s

小さく区切って順次送信

消したいのは、映像を変換し続けるサーバー

01

常時稼働

配信がなくても運用対象

02

トランスコード

CPU / GPUと監視が必要

03

端末で生成

撮影中のiPhoneに任せる

用途を「少人数・短時間・単一品質」に絞る

変えるのは、セグメントを作る場所だけ



— PRECISE WORDING

「サーバーなし」ではなく
「映像変換サーバーなし」

API S3 CDN 状態管理 は残る

1つのsample bufferを、S3公開まで追う

01

HLSの最小形

02

iOSの責務分割

03

Captureと時刻

04

fMP4 segment

05

**S3 uploadと
commit**

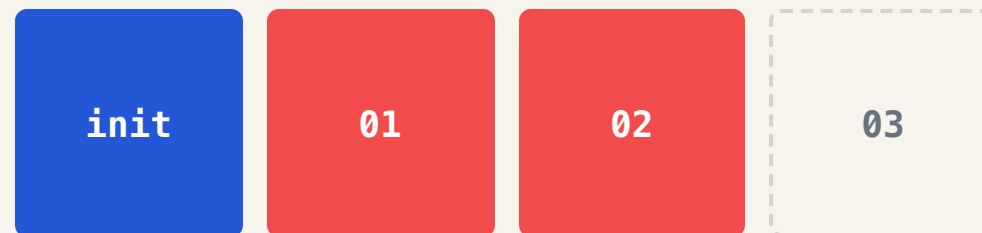
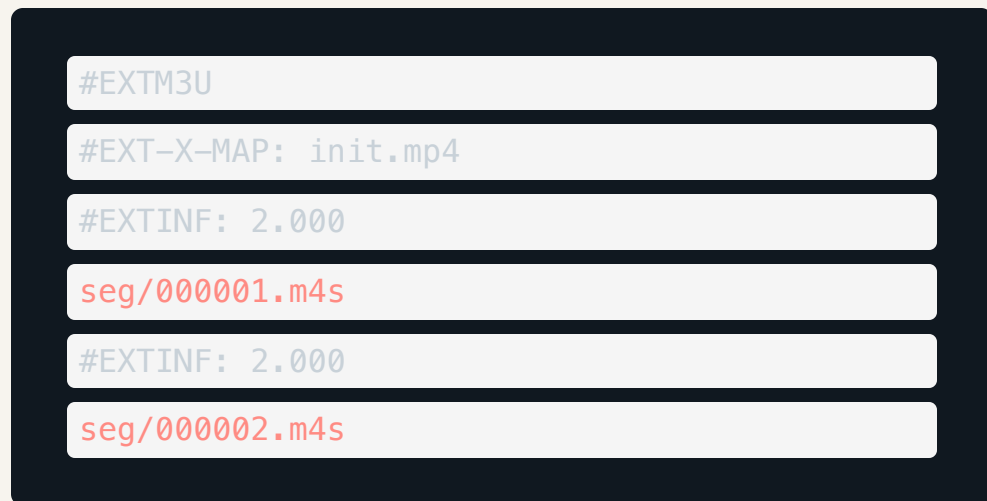
Public at iOSDC 2026 / iosdc2026HLSSample

01

HLSは ファイルと更新される目次

HTTP Live Streaming in the smallest useful model

再生の入口は、動画ファイルではなくplaylist



iPhoneが生成・公開するものは3種類

M3U8

playlist.m3u8

更新される再生順

MP4

init.mp4

codec / track情報

M4S

000001.m4s

2秒前後の映像と音声

playlistが、初期化情報とメディア断片を結びつける

S3では、1つのprefixが1配信

```
live/hls/streams/{streamId}/
```

```
├── init.mp4          1回
├── playlist.m3u8     segmentごとに更新
├── status.json       配信状態
└── seg/
    ├── 000001.m4s
    └── 000002.m4s
```

5つのタグで、再生契約が決まる

```
#EXTM3U
#EXT-X-VERSION:7
#EXT-X-TARGETDURATION:2
#EXT-X-PLAYLIST-TYPE:EVENT
#EXT-X-MAP:URI="init.mp4"
#EXT-X-MEDIA-SEQUENCE:1
```

VERSION

使うタグの互換レベル

TARGETDURATION

segment長の上限基準

EVENT

末尾へ追加するplaylist

MAP

fMP4の初期化セグメント

MEDIA-SEQUENCE

最初の番号

EVENTは「追記」され、最後に一度だけ閉じる

t = 0

header

segmentなし

t = 2s

+ seg 1

追記

t = 4s

+ seg 2

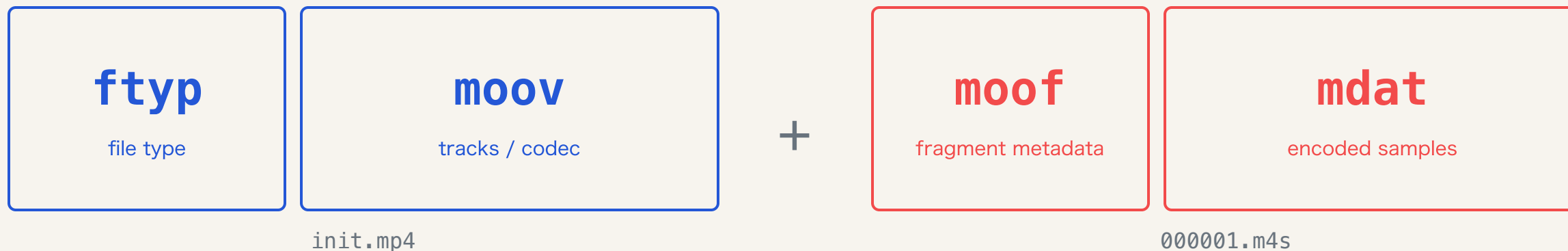
追記

stop

ENDLIST

もう増えない

fMP4は「設定」と「再生可能な断片」を分ける



AVAssetWriterはsegmentを作る。playlistは作らない。

AVFoundation

encode + segment

- H.264 / AAC
- init.mp4
- separable m4s

App

seq + PUT + commit

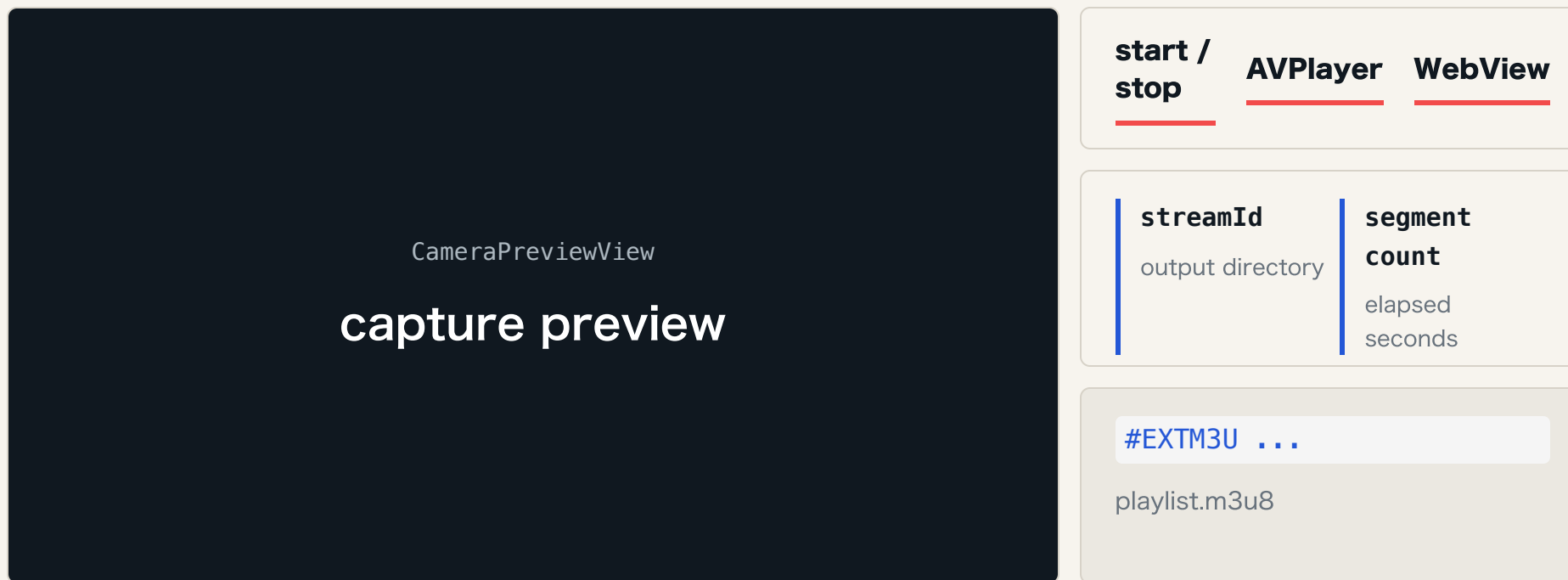
- sequence番号
- S3 upload
- 公開可能なseqを通知

02

公開サンプルで iOSの生成処理だけを拡大する

A public code sample for reading the AVFoundation pipeline

画面上で、カメラ・ファイル・playlistを同時に見る



公開サンプルでは、S3をローカル保存へ置き換える

IN	camera / microphone	AVAssetWriter	OUT	presign / S3 PUT	API / Lambda
	init / media Data	sequence番号		SwiftData	purchase / sharing
	playlist規則			viewer metrics	

本番との差分は、callback後の保存先と公開処理

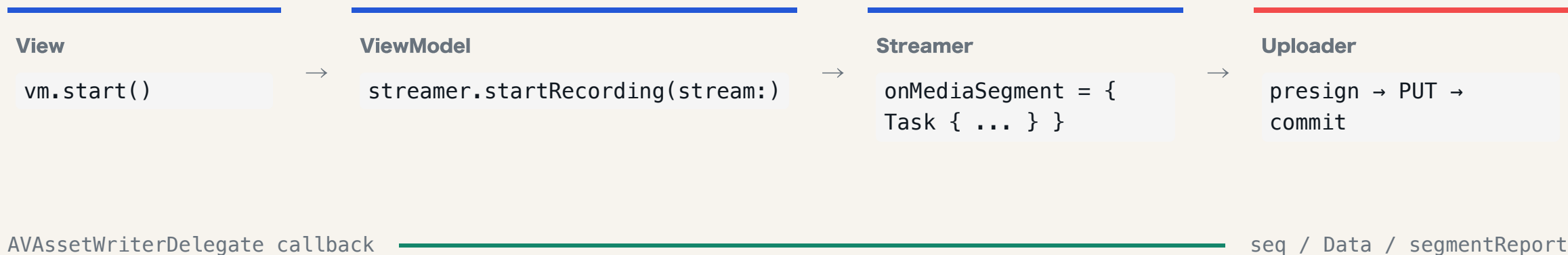
8ファイルで、生成から再生まで完結する

HLSegmentRecorder.swift	capture / encode / segment	LocalHLSStreamer.swift	recorderとstoreを接続
LocalHLSStreamStore.swift	directory / file write	LocalHLSManifest.swift	playlist state
SampleStreamViewModel.swift	permission / UI state	ContentView.swift	操作と観測
LocalHLSPlayerView.swift	AVPlayer	LocalHLSWebPreview.swift	WKWebView

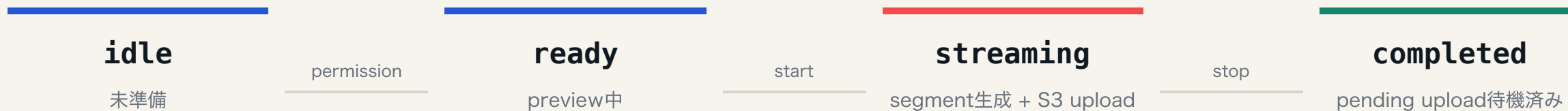
本番は4つの型で、生成と公開を分ける

UI	LiveStreamViewModel	ready / streaming / completed
ORCHESTRATE	LiveHLSStreamer	recorder callbackをuploadへ接続
MEDIA	LiveSegmentRecorder	capture / writer / media clock
UPLOAD	HLSTransmissionCoordinator	presign / PUT / commit

配信開始タップで、RecorderとUploaderを接続する

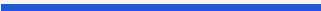


streamingは「生成中かつアップロード中」



どの非同期処理からも `error(String)` へ遷移

Queueとactorは、守る状態が違う

MainActor	ViewModelの表示状態	
sessionQueue	AVCaptureSessionの構成・start / stop	
writingQueue	sample順序・AVAssetWriter状態	
HLSUploadCoordinator actor	upload数・commit状態・終了状態	

1本の巨大なロックではなく、責任ごとに直列化する

03

Captureが メディアの時計を作る

Camera, microphone, sample buffers, and one shared timeline

上り回線に合わせて、配信品質を3段階から選ぶ

```
1 let profile: (CGSize, Int) = switch quality {  
2 case .high:  (.init(width: 720, height: 1280), 2_500_000)  
3 case .medium: (.init(width: 720, height: 1280), 1_500_000)  
4 case .low:   (.init(width: 480, height: 854),   900_000)  
5 }
```

NWPath

constrained / expensive

presign

API latency

probe PUT

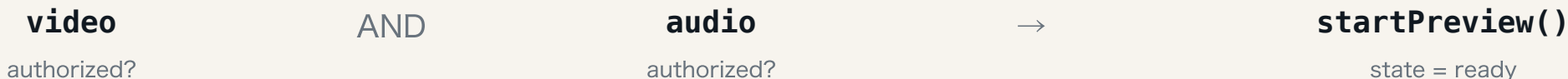
upload throughput

2.0 sec

segment target

— PERMISSION FIRST

権限がそろうまで、CaptureSessionを起動しない



```
1 let cameraAllowed = await requestCameraPermission()
2 let micAllowed = await requestMicrophonePermission()
3 guard cameraAllowed, micAllowed else { ... }
```

— PREVIEW ≠ RECORDING

カメラ起動とHLS生成は、別の操作

PHASE 1

startPreview()

CaptureSessionを構成してstartRunning

then

PHASE 2

startRecording()

AVAssetWriterを準備してisWriting = true

プレビューを見せたまま、Writerだけ開始・終了できる

AVCaptureSessionが、2つのdeviceを2つのDataOutputへつなぐ

Back camera

AVCaptureDeviceInput

Microphone

AVCaptureDeviceInput

AVCaptureSession

VideoDataOutput

CMSampleBuffer

AudioDataOutput

CMSampleBuffer

カメラ構成より先に、録音用AudioSessionを有効化する

```
1 let audio = AVAudioSession.sharedInstance()
2 try audio.setCategory(
3     .playAndRecord,
4     mode: .videoRecording,
5     options: [.defaultToSpeaker, .allowBluetoothHFP]
6 )
7 try audio.setActive(true)
```

Capture

camera + microphone

Route

speaker + Bluetooth HFP

AVCaptureSessionだけでなく、AVAudioSessionのrouteも配信品質に影響する

完成ファイルではなく、sample bufferが欲しい

MovieFileOutput

録画ファイルを作る

高レベルで便利



Video / Audio DataOutput

各sampleを受け取る

Writerへ逐次append

video CMSampleBuffer

audio CMSampleBuffer

same writingQueue

CameraのNV12を、WriterがH.264へ圧縮する

Capture

420f / NV12

pixel buffer

append

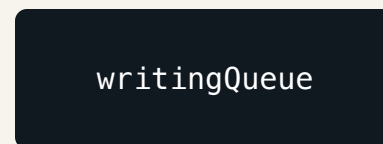
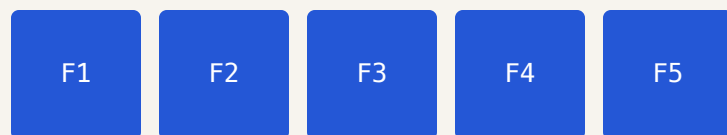
Writer

H.264 High

fMP4 sample

```
1 videoOutput.videoSettings = [  
2     kCVPixelBufferPixelFormatTypeKey as String:  
3     kCVPixelFormatType_420YpCbCr8BiPlanarFullRange  
4 ]
```

遅れたframeは、無制限に溜めない

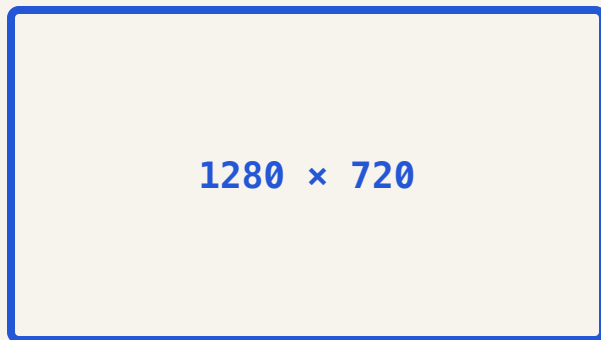


append

late frameをdrop

```
1 videoOutput.alwaysDiscardsLateVideoFrames = true
```

縦向きは、video connectionへ90度で指定する



```
1 if let connection = videoOutput.connection(with: .video),  
2   connection.isVideoRotationAngleSupported(90) {  
3     connection.videoRotationAngle = 90  
4   }
```

Writerへ届く前に、2つのguardで落とす

```
captureOutput(_ : didOutput: from:)
```

```
isWriting == true
```

録画中だけ

```
CMSampleBufferDataIsReady
```

利用可能なsampleだけ

```
startWriterIfNeeded → append
```

```
1 guard isWriting else { return }  
2 guard CMSampleBufferDataIsReady(sampleBuffer) else { return }
```

最初のvideo sampleが、全trackの開始を決める

audio sample

まだappendしない

first video sample

startWriting

startSession(at: 10s)

video + audio共通

```
1 guard output === videoOutput else { return }
2 guard !didStartSession, let writer else { return }
```

Capture PTSは、そのままHLSの開始時刻ではない

Capture PTS

58342.31

...

– firstVideoPTS + 10s

HLS timeline

10.00

...

— ONE DELTA

時刻補正は、全sampleへの平行移動

delta = 10s - firstVideoPTS

adjustedPTS = sourcePTS + delta

Media bytesは変えず、timing metadataをコピーする

```
1  let timingInfos = try sampleTimingInfos().map { info in
2      var adjusted = info
3      adjusted.presentationTimeStamp =
4          info.presentationTimeStamp + offset
5      if info.decodeTimeStamp.isValid {
6          adjusted.decodeTimeStamp = info.decodeTimeStamp + offset
7      }
8      return adjusted
9  }
10
11 let copied = try CMSampleBuffer(copying: self, withNewTiming: timingInfos)
```

PTSと、有効なDTSを同じ量だけ動かす

VideoとAudioを別々に補正しない

video	58342.31	same delta	video	10.00
audio	58342.29		audio	9.98

相対差を保ったまま、Writerのtime rangeへ移す

Writerが詰まったsampleは、待たずに落とす

```
1  if output === videoOutput {  
2      guard videoInput.isReadyForMoreMediaData else { return }  
3      didAppend = videoInput.append(adjustedSampleBuffer)  
4  } else {  
5      guard audioInput.isReadyForMoreMediaData else { return }  
6      didAppend = audioInput.append(adjustedSampleBuffer)  
7  }
```

push source

×

real-time writer

→

bounded latency

— STOP IN THE SAME CLOCK

開始と終了を、同じ補正後timelineで閉じる

lastAdjustedPTS

最後にappendした時刻

→

endSession

media rangeを閉じる

→

markAsFinished

video / audio

→

finishWriting

最後のsegmentをflush

```
1  if lastAdjustedPTS.isValid {  
2      writer.endSession(atSourceTime: lastAdjustedPTS)  
3  }
```

04

AVAssetWriterが sampleをfMP4へ分ける

Four HLS settings and a segment delegate

ファイルURLを渡さず、Dataをdelegateで受け取る

```
1 let writer = AVAssetWriter(contentType: .mpeg4Movie)
2 writer.shouldOptimizeForNetworkUse = true
3 writer.delegate = self
```

AVAssetWriter

MP4 content type

initialization Data

separable Data

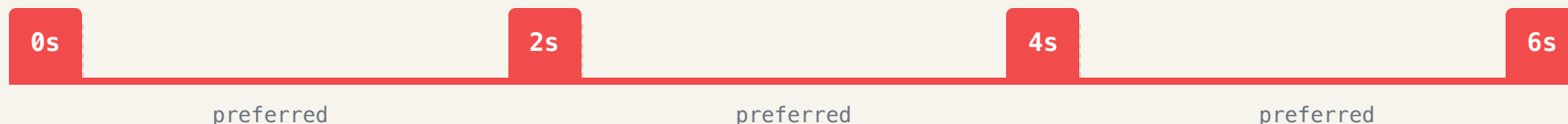
— FOUR HLS KNOBS

HLS出力は、4つのpropertyで有効になる

- | | | |
|----|---|-----------------|
| 01 | <code>outputFileTypeProfile</code> | Apple HLS向けfMP4 |
| 02 | <code>preferredOutputSegmentInterval</code> | 希望するsegment間隔 |
| 03 | <code>initialSegmentStartTime</code> | 最初のsegment開始時刻 |
| 04 | <code>delegate</code> | 生成されたDataの受け口 |

— PREFERRED INTERVAL

2秒は「希望」。実際の境界はmedia条件にも依存する



```
1 writer.preferredOutputSegmentInterval = CMTime(  
2     seconds: config.segmentSeconds,  
3     preferredTimescale: 600  
4 )
```

現在は2.0秒固定。segmentReport反映は改善項目

— INITIAL START TIME

Writerの最初のsegmentを、10秒起点へ固定する



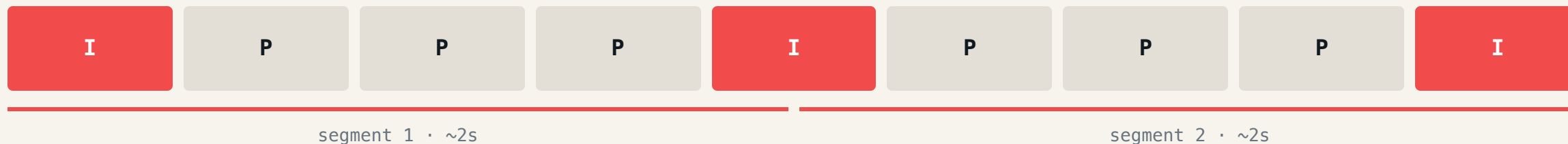
```
1 private let startTimeOffset = CMTime(value: 10, timescale: 1)
2 writer.initialSegmentStartTime = startTimeOffset
```

配信用videoは、互換性と上り帯域を優先する

KEY	VALUE	INTENT
AVVideoCodecKey	H.264	広い再生互換性
Width / Height	480p / 720p	quality別を選択
AverageBitRate	0.9-2.5 Mbps	上り回線へ追従
ProfileLevel	High Auto	encoder profile
FrameReordering	false	decode順を単純化

— KEYFRAME ALIGNMENT

2秒で切るなら、2秒以内にIDRを用意する



```
1 AVVideoMaxKeyFrameIntervalDurationKey: config.segmentSeconds
```

AudioはAAC mono、品質別に48-96kbps

FORMAT	CHANNEL	SAMPLE RATE	BITRATE
AAC	mono	44.1 kHz	48-96 kbps

```
1  [
2    AVFormatIDKey: kAudioFormatMPEG4AAC,
3    AVNumberOfChannelsKey: 1,
4    AVSampleRateKey: 44_100,
5    AVEncoderBitRateKey: config.audioBitrate
6  ]
```

segmentTypeだけで、initとmediaを分けられる

```
1 func assetWriter(  
2     _ writer: AVAssetWriter,  
3     didOutputSegmentData segmentData: Data,  
4     segmentType: AVAssetSegmentType,  
5     segmentReport: AVAssetSegmentReport?  
6 ) {  
7     switch segmentType {  
8     case .initialization:  
9         onInitSegment?(segmentData)  
10    case .separable:  
11        segmentIndex += 1  
12        onMediaSegment?(segmentIndex, segmentData, segmentReport)  
13    @unknown default:  
14        break  
15    }  
16 }
```

その後のcallbackは、番号付きm4sへ変わる

segmentType

.separable



Data

moof + mdat

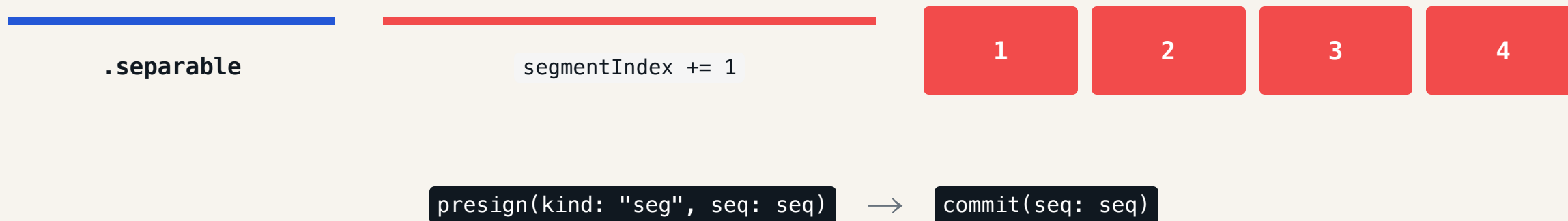


presigned PUT

S3 / seg / seq=1

Dataを再変換せず、そのまま保存またはuploadできる

再生順の番号は、delegate callbackごとに自分で付ける



現在は2.0秒固定。segment reportは次の精度改善点

CURRENT

durationSec = 2.0

fragmentSecondsをcommit

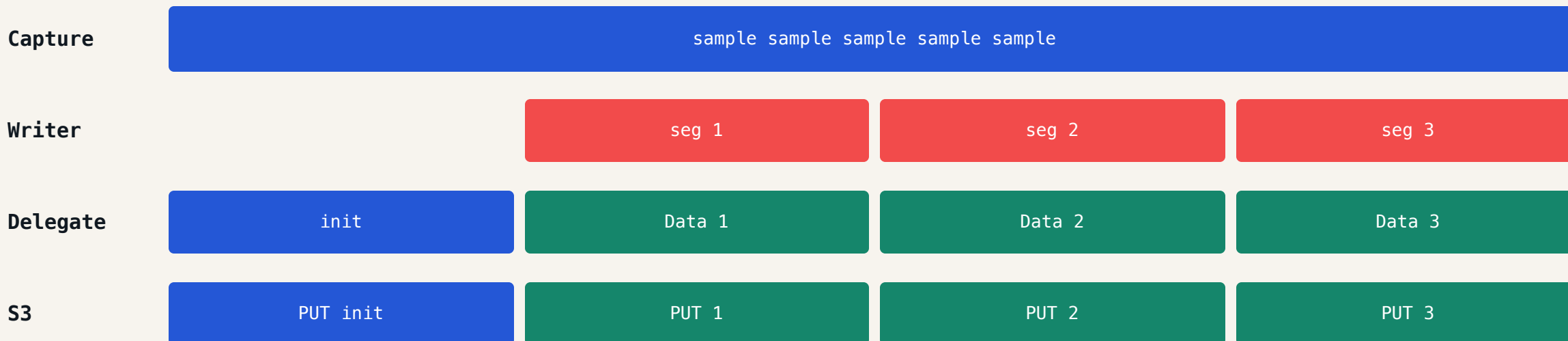
NEXT

segmentReportから実測

EXTINFへ正確な長さ

```
1 recorder.onMediaSegment = { [uploader, fragmentSeconds] seq, data, _ in
2     try? await uploader.uploadSegment(
3         stream: stream, seq: seq, segmentData: data,
4         durationSec: fragmentSeconds
5     )
6 }
```

Captureを止めず、2秒前後ごとにDataが外へ出る



Writerのfinishを待たず、生成とS3 uploadを並行できる

Writerが詰まっても、Captureを待たせない

DataOutput

real-time push

`isReadyForMoreMediaData`

true

append

false

return / drop

長所: latencyとmemoryがbounded

代償: frame dropを観測していない

最初の再生可能状態は、init PUTとsegment commitの後

start	network	~2s	after PUT
init Data	PUT init	PUT seg 1	commit seq 1

init object exists
playlist contains seq 1

seg 1 object exists
Viewer can start

05

端末で生成したDataを S3へ順番に公開する

Presign, PUT, commit, and finish from the iOS point of view

segmentが来る前に、streamIdと再生URLを準備する

createStream

streamId / prefix

→

ticket

playbackUrl

→

permission

camera / microphone

→

ready

previewSession

```
1  if ticket == nil {  
2      ticket = try await apiClient.ticket(streamId: stream.streamId)  
3  }  
4  await requestPermissionsAndSetup()  
5  try await streamer?.start()
```

iOS側は「生成」「接続」「公開」を分ける

MEDIA

LiveSegmentRecorder

Data callbackを生成

BRIDGE

LiveHLSStreamer

callbackをTaskへ変換

UPLOAD

HLSUploadCoordinator

presign / PUT / commit

HTTP

HLSAPIClient

URLSessionとAPI contract

AVFoundationのqueueへネットワーク待ちを持ち込まない

init.mp4は、最初のcallbackで1回だけPUTする

```
1 recorder.onInitSegment = { [weak self] data in
2     Task {
3         try? await self?.uploader.uploadInitIfNeeded(
4             stream: stream, initData: data
5         )
6     }
7 }
8
9 guard !didUploadInit else { return }
10 let presign = try await api.presign(
11     streamId: stream.streamId, kind: "init", seq: nil
12 )
13 try await api.put(to: URL(string: presign.putUrl)!, data: initData,
14     contentType: "video/mp4")
15 didUploadInit = true
```

media Dataごとに、presign → PUT → commit

```
1  let presign = try await api.presign(  
2    streamId: stream.streamId,  
3    kind: HLSUploadKind.segment,  
4    seq: seq  
5  )  
6  try await api.put(  
7    to: URL(string: presign.putUrl!),  
8    data: segmentData,  
9    contentType: HLSContentType.segmentM4S  
10 )  
11 _ = try await api.commit(  
12   streamId: stream.streamId, seq: seq,  
13   durationSec: dur, isLast: false,  
14   targetDurationSec: targetDurationSec  
15 )
```

iPhoneへ渡すのは、短時間だけ有効なPUT URL

DO NOT

AWS credentialを内包

漏えい範囲が広い



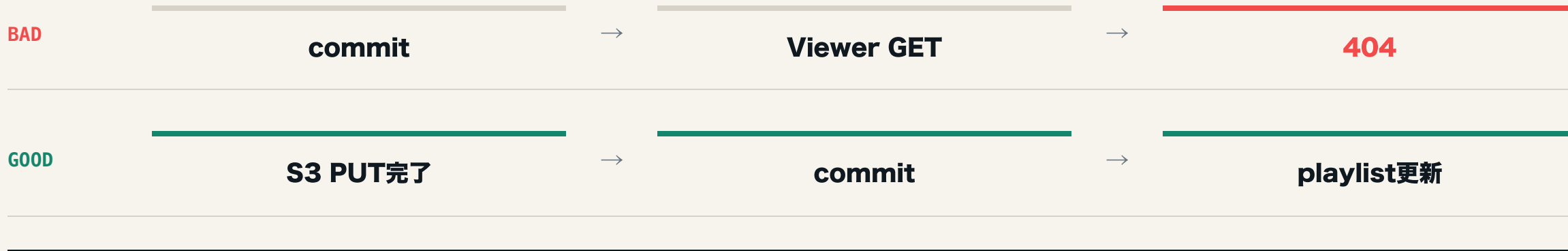
PRESIGNED URL

object単位のPUT権限

streamId / kind / seqで制約

```
1 var request = URLRequest(url: presignedURL)
2 request.httpMethod = "PUT"
3 request.setValue(contentType, forHTTPHeaderField: "Content-Type")
4 request.httpBody = data
```

PUT成功より先に、playlistへseqを出さない



存在するsegmentだけを、再生可能として公開する

Actorは、uploadと終了の状態をまとめて守る

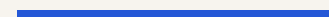
didUploadInit

init.mp4の重複PUTを防ぐ



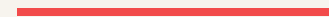
lastCommittedSeq

公開済み最大seq



inUploadSegmentCount

停止時に待つbacklog



isFinishing

遅れて完了したsegmentでもENDLISTを再通知



actor内のawaitは再入可能。完了順ではなくseqを正とする

stop後は pending uploadが0になるまで完了にしない



```
1 while await streamer.hasPendingUploads() {  
2     try? await Task.sleep(for: .milliseconds(500))  
3 }
```

公開サンプルと本番、違うのは保存先

SAMPLE	PRODUCTION iOS	VIEWER
Data.write init / m4sを直接観察	presigned PUT 同じDataをS3へ送る	CloudFront URL commit済みseqを再生

Data.write と presigned PUT。違うのはcallback後だけ

06

iOSの非同期境界を ライブ配信として閉じる

Callback, task, upload backlog, and publication order

callbackでは待たず、TaskへDataを引き渡す

LiveSegmentRecorder

init Data / media Data

WRITING QUEUE

callbackを返す

次のsample処理へ

SWIFT TASK

Uploaderをawait

presign / PUT / commit

initialization → uploadInitIfNeeded

separable → uploadSegment

segmentReport → duration改善余地

— THREE REQUESTS

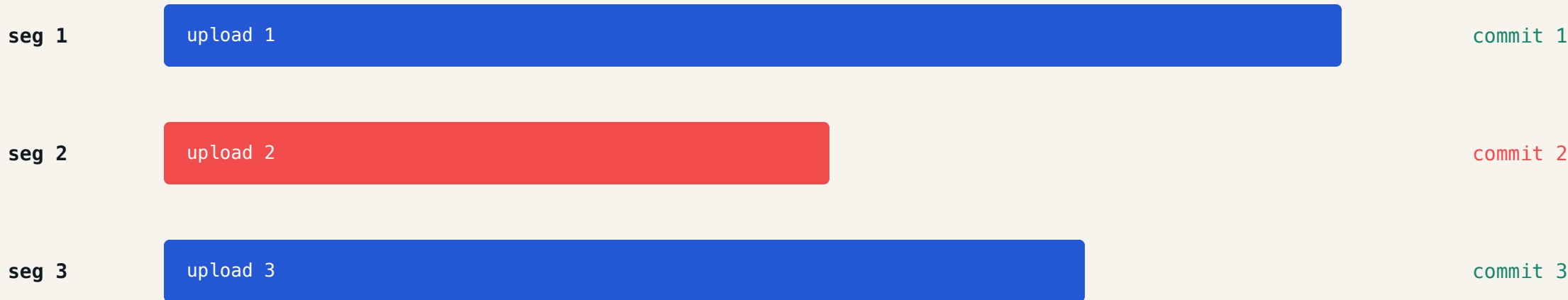
presign・PUT・commitは、役割が違う

REQUEST	DESTINATION	ROLE
POST /presign	API	seq用PUT URLを発行
PUT putUrl	S3	video/mp4 Dataを保存
POST /commit	API	seqをplaylistへ公開
GET playbackUrl	CloudFront	Viewerがplaylistを取得

iOSはplaylist本文を書かず、「公開してよいseq」をcommitする

— OUT-OF-ORDER RACE

2秒ごとのuploadは、完了順まで保証しない



completion 2, 1, 3 $\neq$ playback 1, 2, 3

commit APIは、欠番より先をplaylistへ出さない契約が必要

iPhone側で、upload backlogと速度を測る

METRIC	MEANING	USE
pendingSegmentCount	未完了upload数	stop待機 / backlog
averageRoundTripSec	presign + PUT	segment間隔と比較
averageUploadDurationSec	PUT所要時間	回線劣化を検知
averageUploadThroughputBps	送信bitrate	品質選択の根拠

平均は smoothingFactor = 0.25 の指数移動平均

— FIT, NOT UNIVERSAL

この構成が効くのは、用途を絞ったとき

GOOD FIT

少人数・短時間・単一品質

新しいiPhone / 安定した上り回線

USE A MEDIA PLATFORM

大規模・長時間・ABR・厳しいSLA

発熱、電池、回線変動を吸収したい

「端末でできる」と「端末でやるべき」は別の判断

— TAKEAWAYS

成立条件は、4つの順序をそろえること

01

Clock

Capture PTSをWriterのtimelineへ移す

02

Boundary

IDRとsegment intervalをそろえる

03

Upload

DataをS3へPUTしてからcommitする

04

Finish

pendingが0になるまで完了にしない

ありがとうございました

zenn.dev/hs7/articles/080eac650f65ba

